Rapport Labo 5 - Computer Architectuur

Victor Deforce en Cían Olieslagers

March 16, 2025

1 Probleemstelling

De opgave bestaat uit het uitvoeren van reduction-operaties (som, product, minimum en maximum) op vier onafhankelijke arrays. Dit gebeurt op twee manieren:

- **Approach 1: Synchrone uitvoering**
  Elke array wordt volledig verwerkt voordat de volgende start. Dit omvat het genereren, kopiëren naar de GPU, uitvoeren en terugkopiëren naar de CPU.
- **Approach 2: Asynchrone uitvoering**
  De CPU genereert nieuwe arrays terwijl de GPU berekeningen uitvoert, wat parallelle verwerking mogelijk maakt en idle tijd vermindert.

Het doel is om de totale uitvoeringstijd van beide methoden te vergelijken.

2 Implementatie

2.1 Sequential execution

We hebben onze implementatie uit 4 analoge blokken opgebouwd. In deze blokken callen we de Kernel die door middel van reduction het minimum, het maximum, het product of de som van alle elementen uit de array bepaald. Verder callen we cudaDeviceSynchronise() na iedere kernel zodat deze elkaar sequentieel opvolgen. De uitvoeringstijd van de vier kernels met de aanmaak van de array meten we op aan het einde van elke iteratie en we herhalen deze meting 100 keer om de variantie op de resultaten te beperken.

2.2 Parallel execution

Deze implementatie verloopt analoog. Om de kernels echter asynchroon te laten werken vervangen we de cudaMemcpy door cudaMemcpyAsync en synchroniseren we enkel na de laatste kernel. Zo kan de CPU verder zonder op de GPU te wachten.

3 Observatie

Uit Figuur 1 blijkt dat, ongeacht de grootte van de array, de asynchrone uitvoering aanzienlijk sneller is dan de synchrone uitvoering. Dit komt doordat de CPU niet hoeft te wachten op de voltooiing van een kernel voordat de volgende array wordt gegenereerd.
In plaats daarvan kan de CPU direct na een kernel-call beginnen met het voorbereiden van de volgende array, terwijl bij de synchrone uitvoering de CPU telkens moet wachten tot de vorige kernel-call volledig is afgerond. Verder zagen we in labo 1 dat voor kleine arrays de CPU altijd sneller is doordat het gebruik van de GPU extra overhead met zich meebrengt. Dit heeft tot gevolg dat de Kernels quasi volledig parallel worden uitgevoerd in deze opstelling. Dit verklaart ook waarom bij arrays van 2048 elementen de looptijd stijgt ten op zichtte van de kortere arrays. Bij deze grootte is de looptijd van de CPU niet meer verwaarloosbaar ten op zichtte van de looptijd van de GPU en is dus de tijdsduur tussen de start van de eerste kernel en de start van de laatste ook niet meer te verwaarlozen.

4 Conclusie

De resultaten van het verslag tonen aan dat de asynchrone uitvoering aanzienlijk efficiënter is dan de synchrone methode. Dit komt doordat de CPU en GPU parallel kunnen werken, waardoor wachttijden worden geminimaliseerd. In de synchrone uitvoering moet de CPU wachten op de voltooiing van elke kernel-call voordat de volgende array wordt gegenereerd, terwijl de asynchrone aanpak dit probleem omzeilt door gegevensoverdracht en berekeningen simultaan te laten verlopen.

5 Appendix - code

5.1 Approach 1: Sequential execution

```
#include <chrono>
#include <stdio.h>

__global__ void maxReductionSyncKernel(int *array, int size) {
  int roof = size>2048 ? 2*blockDim.x : size;
  for(int spil = 2; spil<=roof; spil=spil*2){
    int idx = (threadIdx.x * spil + blockIdx.x * blockDim.x);
    if (idx+spil/2<((1+blockIdx.x) * 2 * blockDim.x)) {
      if (array[idx] < array[idx + spil / 2]) {
        array[idx] = array[idx + spil / 2];
      }
    }
    __syncthreads();
  }
}

__global__ void minReductionSyncKernel(int *array, int size) {
  int roof = size>2048 ? 2*blockDim.x : size;
  for(int spil = 2; spil<=roof; spil=spil*2){
    int idx = (threadIdx.x * spil + blockIdx.x * blockDim.x);
    if (idx+spil/2<((1+blockIdx.x) * 2 * blockDim.x)) {
      if (array[idx] > array[idx + spil / 2]) {
        array[idx] = array[idx + spil / 2];
      }
    }
    __syncthreads();
  }
}

__global__ void sumReductionSyncKernel(int *array, int size) {
  int roof = size>2048 ? 2*blockDim.x : size;
  for(int spil = 2; spil<=roof; spil=spil*2){
    int idx = (threadIdx.x * spil + blockIdx.x * blockDim.x);
    if (idx+spil/2<((1+blockIdx.x) * 2 * blockDim.x)) {
      array[idx] = array[idx] + array[idx + spil / 2];
    }
    __syncthreads();
  }
}

__global__ void prodReductionSyncKernel(int *array, int size) {
  int roof = size>2048 ? 2*blockDim.x : size;
  for(int spil = 2; spil<=roof; spil=spil*2){
    int idx = (threadIdx.x * spil + blockIdx.x * blockDim.x);
    if (idx+spil/2<((1+blockIdx.x) * 2 * blockDim.x)) {
      array[idx] = array[idx] * array[idx + spil / 2];
    }
    __syncthreads();
  }
}

int main() {
  FILE *file = file = fopen("output_lab5.csv", "w");
```

```
if (!file) {
  perror("File opening failed");
  exit(EXIT_FAILURE);
} else {
  fprintf(stderr, "Writing output to file %s\n", "output.txt");
}
const int iterations = 2000;
for(int k = 2; k<= 2049; k = k*2){
const int size = k;
{
  for (int j = 0; j < iterations; j++) {

    auto start = std::chrono::steady_clock::now();

    {
    // SUM
    int* h_a = new int[size];
    for(int i = 0; i < size; i++){
      h_a[i] = std::rand();
    }
    int *d_a;

    cudaMalloc((void **)&d_a, size * sizeof(int));
    cudaMemcpy(d_a, h_a, size * sizeof(int), cudaMemcpyHostToDevice);

    sumReductionSyncKernel<<<1, (size/2)>>>(d_a, size);
    cudaDeviceSynchronize();

    cudaMemcpy(h_a, d_a, size * sizeof(int), cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    free(h_a);
    }

    {
    // PROD
    int* h_a = new int[size];
    for(int i = 0; i < size; i++){
      h_a[i] = std::rand();
    }
    int *d_a;

    cudaMalloc((void **)&d_a, size * sizeof(int));
    cudaMemcpy(d_a, h_a, size * sizeof(int), cudaMemcpyHostToDevice);

    prodReductionSyncKernel<<<1, (size/2)>>>(d_a, size);
    cudaDeviceSynchronize();

    cudaMemcpy(h_a, d_a, size * sizeof(int), cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    free(h_a);
    }

    {
    // MAX
```

```
      int* h_a = new int[size];
      for(int i = 0; i < size; i++){
        h_a[i] = std::rand();
      }
      int *d_a;

      cudaMalloc((void **)&d_a, size * sizeof(int));
      cudaMemcpy(d_a, h_a, size * sizeof(int), cudaMemcpyHostToDevice);

      maxReductionSyncKernel<<<1, (size/2)>>>(d_a, size);
      cudaDeviceSynchronize();

      cudaMemcpy(h_a, d_a, size * sizeof(int), cudaMemcpyDeviceToHost);

      cudaFree(d_a);
      free(h_a);
      }

      {
      // MIN
      int* h_a = new int[size];
      for(int i = 0; i < size; i++){
        h_a[i] = std::rand();
      }
      int *d_a;

      cudaMalloc((void **)&d_a, size * sizeof(int));
      cudaMemcpy(d_a, h_a, size * sizeof(int), cudaMemcpyHostToDevice);

      minReductionSyncKernel<<<1, (size/2)>>>(d_a, size);
      cudaDeviceSynchronize();

      cudaMemcpy(h_a, d_a, size * sizeof(int), cudaMemcpyDeviceToHost);

      cudaFree(d_a);
      free(h_a);
      }

      long duration = std::chrono::duration_cast<std::chrono::microseconds>(
        std::chrono::steady_clock::now() - start)
        .count();

      fprintf(file, "%lu,", duration);
    }
  }
  fprintf(file, "\n");
}

  return 0;
}
```

5.2 Approach 2: Parallel execution

```
#include <chrono>
#include <stdio.h>

//Kernels blijven hetzelfde

int main() {
  FILE *file = file = fopen("output_lab5.1.csv", "w");
  if (!file) {
    perror("File opening failed");
    exit(EXIT_FAILURE);
  } else {
    fprintf(stderr, "Writing output to file %s\n", "output.txt");
  }
  const int iterations = 2000;
  for(int k = 2; k<= 2049; k = k*2){
  const int size = k;
  {
    for (int j = 0; j < iterations; j++) {

      auto start = std::chrono::steady_clock::now();

      // SUM
      int* h_a1 = new int[size];
      for(int i = 0; i < size; i++){
        h_a1[i] = std::rand();
      }
      int *d_a1;

      cudaMalloc((void **)&d_a1, size * sizeof(int));
      cudaMemcpyAsync(d_a1, h_a1, size * sizeof(int), cudaMemcpyHostToDevice);

      sumReductionSyncKernel<<<1, (size/2)>>>(d_a1, size);

      cudaMemcpyAsync(h_a1, d_a1, size * sizeof(int), cudaMemcpyDeviceToHost);

      // PROD
      int* h_a2 = new int[size];
      for(int i = 0; i < size; i++){
        h_a2[i] = std::rand();
      }
      int *d_a2;

      cudaMalloc((void **)&d_a2, size * sizeof(int));
      cudaMemcpyAsync(d_a2, h_a2, size * sizeof(int), cudaMemcpyHostToDevice);

      prodReductionSyncKernel<<<1, (size/2)>>>(d_a2, size);

      cudaMemcpyAsync(h_a2, d_a2, size * sizeof(int), cudaMemcpyDeviceToHost);

      // MAX
      int* h_a3 = new int[size];
      for(int i = 0; i < size; i++){
        h_a3[i] = std::rand();
      }
      int *d_a3;
```

```
      cudaMalloc((void **)&d_a3, size * sizeof(int));
      cudaMemcpyAsync(d_a3, h_a3, size * sizeof(int), cudaMemcpyHostToDevice);

      maxReductionSyncKernel<<<1, (size/2)>>>(d_a3, size);

      cudaMemcpyAsync(h_a3, d_a3, size * sizeof(int), cudaMemcpyDeviceToHost);

      // MIN
      int* h_a4 = new int[size];
      for(int i = 0; i < size; i++){
        h_a4[i] = std::rand();
      }
      int *d_a4;

      cudaMalloc((void **)&d_a4, size * sizeof(int));
      cudaMemcpyAsync(d_a4, h_a4, size * sizeof(int), cudaMemcpyHostToDevice);

      minReductionSyncKernel<<<1, (size/2)>>>(d_a4, size);

      cudaMemcpyAsync(h_a4, d_a4, size * sizeof(int), cudaMemcpyDeviceToHost);

      cudaFree(d_a4);
      free(h_a4);

      cudaFree(d_a3);
      free(h_a3);

      cudaFree(d_a2);
      free(h_a2);

      cudaFree(d_a1);
      free(h_a1);

      cudaDeviceSynchronize();

      long duration = std::chrono::duration_cast<std::chrono::microseconds>(
        std::chrono::steady_clock::now() - start)
        .count();

      fprintf(file, "%lu,", duration);
    }
  }
  fprintf(file, "\n");
}

  return 0;
}
```

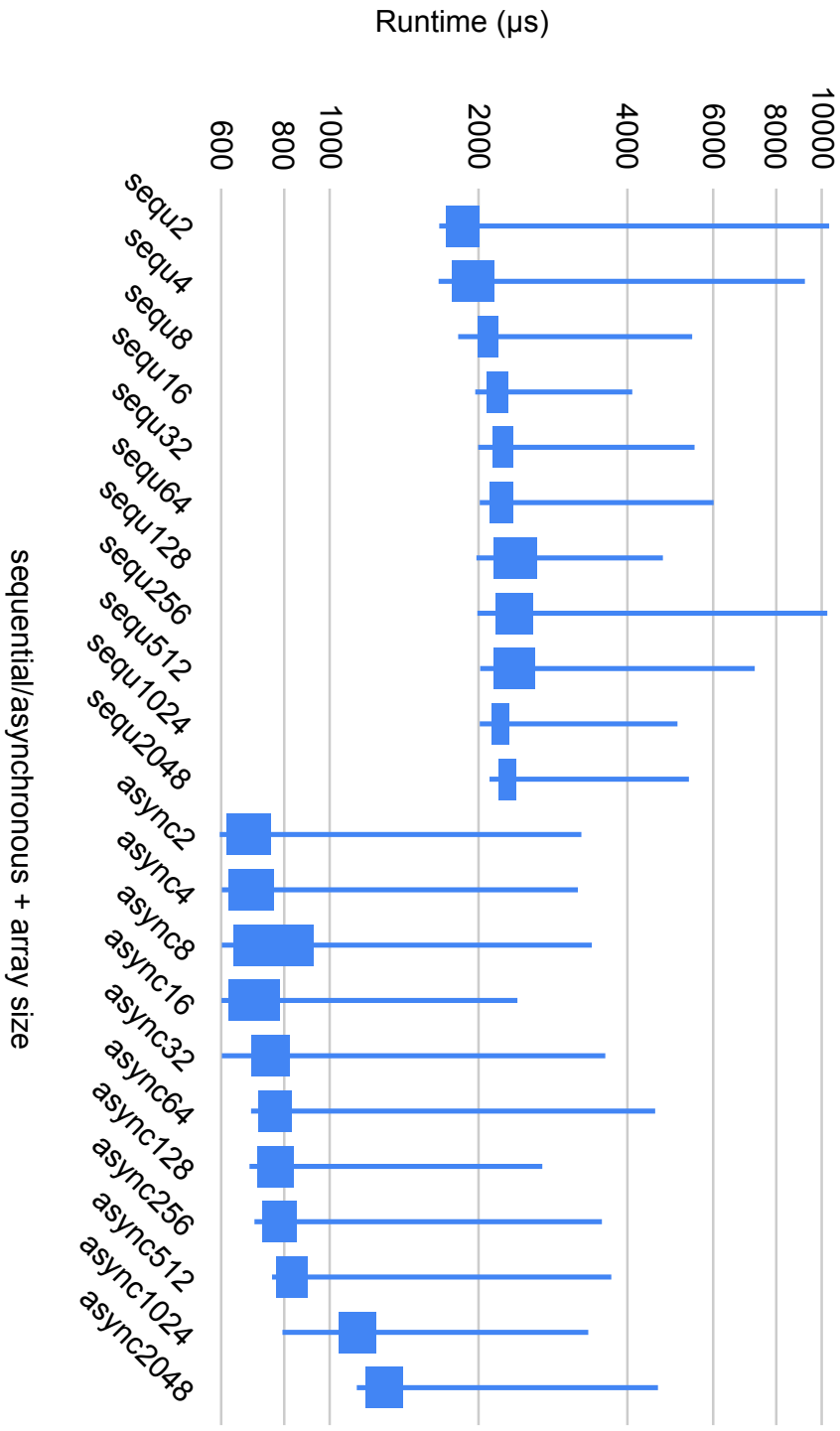

Runtime (µs)
600
800
1000
2000
4000
6000
8000
10000
sequ2
sequ4
sequ8
sequ16
sequ32
sequ64
sequ128
sequ256
sequ512
sequ1024
sequ2048
async2
async4
async8
async16
async32
async64
async128
async256
async512
async1024
async2048
sequential/asynchronous + array size

Figure 1: